Del pipeline al procesador dual-issue

Diseño de un RISC-V superescalar de dos vías, en orden

Un procesador segmentado de cinco etapas puede completar, como máximo, una instrucción por ciclo. Para superar ese límite hay que ejecutar varias instrucciones a la vez. Partimos de un pipeline RISC-V de cinco etapas con manejo completo de hazards y lo transformamos, paso a paso, en un procesador *superescalar dual-issue en orden*, capaz de buscar, decodificar y ejecutar hasta *dos* instrucciones por ciclo.

A partir de la sección 3, cada sección agrega una parte al datapath y presenta la figura resultante, hasta llegar al procesador completo de la Fig. 14. Algunos bloques se detallan con fragmentos de Verilog (los *Ejemplos HDL*). Los diagramas de ejecución de las secciones 5 a 7 provienen de simular ese mismo hardware.

1 El punto de partida: el pipeline de cinco etapas

La Fig. 1 muestra el procesador de partida [1]. Las instrucciones avanzan por cinco etapas: *Fetch* (F), *Decode* (D), *Execute* (E), *Memory* (M) y *Writeback* (W). Las etapas están separadas por registros de pipeline, y cada señal lleva como sufijo la etapa a la que pertenece: *RdE*, por ejemplo, es el registro destino de la instrucción que se encuentra en Execute.

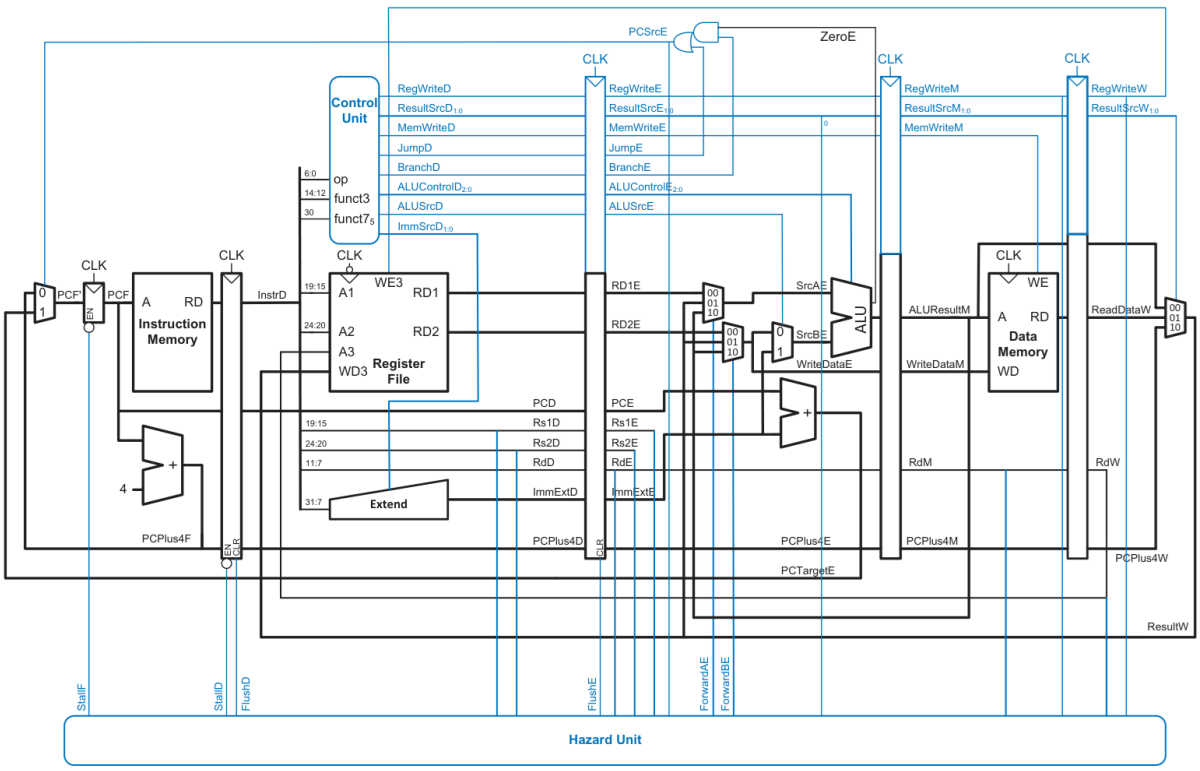


Figura 1 **Procesador segmentado de cinco etapas con manejo de hazards (adaptado de [1])**

Como hay hasta cinco instrucciones en el pipeline al mismo tiempo, aparecen *hazards*: situaciones en las que una instrucción necesita un resultado que otra todavía no ha producido. La Hazard Unit, en la parte inferior de la figura, los resuelve con tres mecanismos:

- **Forwarding.** Si una instrucción en Execute lee un registro que una instrucción anterior aún no ha escrito, el valor se toma directamente de la etapa Memory (*ALUResultM*) o Writeback

(*ResultW*). *ForwardAE* y *ForwardBE* seleccionan la fuente; si ambas etapas contienen el registro, tiene prioridad Memory, porque guarda el valor más reciente.

- **Stall por load-use.** El dato de un `lw` solo está disponible al final de la etapa Memory. Si la instrucción siguiente lo usa, el forwarding no basta: *StallF* y *StallD* detienen Fetch y Decode durante un ciclo, y *FlushE* inserta una burbuja en Execute.
- **Flush por salto.** Los saltos se resuelven en Execute y se predicen como no tomados. Si el salto se toma (*PCSrcE*), las dos instrucciones buscadas a continuación pertenecen al camino incorrecto y se descartan con *FlushD* y *FlushE*.

Además, el register file se escribe en la primera mitad del ciclo y se lee en la segunda, de modo que una instrucción en Decode lee el valor que escribe otra que se encuentra en Writeback.

Aun con estos mecanismos, el pipeline completa como máximo una instrucción por ciclo. En términos del IPC (*instructions per cycle*, el promedio de instrucciones completadas por ciclo), esto significa que IPC ≤ 1 o, de forma equivalente, CPI ≥ 1.

2 Visión general

El pipeline aprovecha el *paralelismo temporal*: varias instrucciones ocupan etapas distintas al mismo tiempo, pero como máximo una entra a Execute en cada ciclo. Un procesador superescalar agrega *paralelismo espacial*: replica el hardware de ejecución para que varias instrucciones avancen juntas por la misma etapa. El procesador que diseñamos es de dos vías (*two-way*) y *en orden*: las instrucciones se emiten en el orden del programa y una instrucción más joven nunca se adelanta a una más antigua. Con dos vías, el IPC ideal es 2.

Terminología. En adelante usamos estos términos:

- **Par**: las dos instrucciones consecutivas que se buscan y decodifican juntas.
- **Slot 0 / slot 1**: posición dentro del par. El slot 0 contiene siempre la instrucción *más antigua* (dirección PCD_0) y el slot 1 la *más joven* ($PCD_1 = PCD_0 + 4$).
- **Vía** (*lane*): cada una de las dos copias del camino de ejecución (Execute, Memory y Writeback). El slot 0 entra siempre a la vía 0 y el slot 1 a la vía 1; por lo tanto, **la vía 1 contiene siempre la instrucción más joven**. Todas las prioridades del diseño se derivan de esta propiedad.
- **Emitir** (*issue*): pasar una instrucción de Decode a Execute. En cada ciclo se emiten dos instrucciones, una o ninguna.

Decisiones de diseño. No conviene replicar todos los recursos. Una ALU es barata, mientras que una memoria de datos de dos puertos o una segunda lógica de resolución de saltos son caras y se usarían poco: en un programa típico, rara vez dos instrucciones consecutivas acceden ambas a memoria o son ambas saltos. Por eso fijamos los recursos de la Tabla 1: casi todo el datapath se replica, pero la memoria de datos y la *Branch Unit*, que resuelve los saltos, son *compartidas*. Al compartirlas, hace falta una unidad que decida, en cada ciclo, qué instrucciones se emiten.

Diagramas de ejecución. La ejecución se representa con diagramas como el de la Fig. 2. Cada fila corresponde a una instrucción y cada columna a un ciclo. Cada celda indica el componente que la instrucción usa en ese ciclo: memoria de instrucciones (IM), register file (RF), ALU, memoria de datos (DM) y, de nuevo, register file en la escritura. La mitad sombreada del RF indica si se lee (segunda mitad del ciclo, en Decode) o se escribe (primera mitad, en Writeback). Las flechas azules representan forwarding, y la columna `v0/v1` indica la vía en la que se ejecuta cada instrucción.

Tabla 1 **Recursos del procesador dual-issue**

Recurso	Cantidad	Consecuencia
Lectura de instrucciones	2 por ciclo	memoria de instrucciones con dos puertos de lectura
Register file	4 lecturas, 2 escrituras	un par completo lee y escribe en un ciclo
ALU	2	dos operaciones ALU por ciclo
Memoria de datos	1 puerto	como máximo un `lw`/`sw` por par
Branch Unit	1	como máximo un `beq`/`jal` por par
Emisión	en orden	la más joven nunca se adelanta a la más antigua

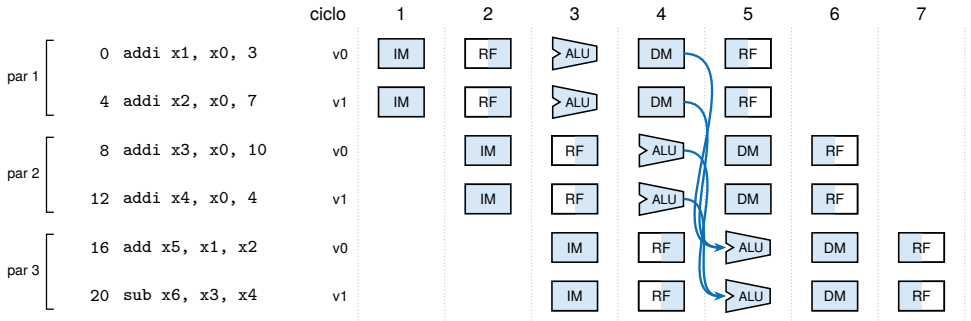


Figura 2 **Tres pares emitidos en ciclos consecutivos; en el ciclo 5 las dos ALU operan simultáneamente y el forwarding cruza entre vías**

La Fig. 2 muestra el caso ideal: tres pares sin dependencias dentro de cada par entran a Execute en ciclos consecutivos. En el ciclo 5, `add x5` (vía 0) recibe `x1` de su propia vía y `x2` de la vía 1, mientras que `sub x6` (vía 1) recibe `x3` de la vía 0. Es decir, el forwarding ahora *cruza entre vías* (sección 5).

El hardware se construye en cuatro pasos:

1. replicar el datapath y compartir la memoria de datos y la Branch Unit (secciones 3 y 4);
2. extender el forwarding a las dos vías (sección 5);
3. agregar la *Issue Unit*, que decide cuántas instrucciones del par se emiten, y el mecanismo de reagrupamiento (sección 6);
4. adaptar los stalls por `lw` y los flushes por salto (sección 7).

Las figuras del datapath usan cinco colores: negro para los datos, azul para el control, naranja para la Hazard Unit y sus señales, verde para la Issue Unit y morado para la Branch Unit. Un punto indica una unión entre cables; dos cables que se cruzan sin punto no están conectados.

3 Datapath de dos vías

El primer paso consiste en construir el datapath de dos vías sin considerar aún los hazards. La Fig. 3 muestra el resultado. La vía 0 (arriba) es el pipeline de la Fig. 1 sin la Hazard Unit, y la vía 1 (abajo) es su imagen especular. Los elementos que comparten ambas vías se ubican sobre el eje central de la figura: la memoria de instrucciones, el registro IF/ID, el register file, la memoria de datos y la Branch Unit.

3.1 Fetch doble

La memoria de instrucciones tiene dos puertos de lectura que reciben la misma dirección *PCF*: $RD1 = \text{mem}[PCF]$ entrega $InstrF_0$ y $RD2 = \text{mem}[PCF + 4]$ entrega $InstrF_1$. Como se buscan dos

instrucciones por ciclo, el PC avanza en 8 (*PCPlus8F*) en lugar de 4.

Un único registro IF/ID de 192 bits almacena el par completo: las dos instrucciones ($InstrD_0$, $InstrD_1$), sus direcciones ($PCD_0 = PCF$, $PCD_1 = PCPlus4F$) y el $PC + 4$ de cada una ($PCPlus4D_0 = PCPlus4F$, $PCPlus4D_1 = PCPlus8F$), que se calculan en Fetch y viajan con el par. Como es un solo registro, el par avanza o se detiene completo, y así se mantiene el orden entre sus dos instrucciones.

3.2 Decode doble y register file

Cada slot tiene su propia unidad de control y su propio bloque Extend. Su $PC + 4$ ($PCPlus4D_k$) no se calcula en Decode: llega ya hecho desde el registro IF/ID. El register file debe atender al par completo en un solo ciclo:

- **cuatro puertos de lectura**: A1/A2 para el slot 0 y A4/A5 para el slot 1;
- **dos puertos de escritura**: A3/WD3/WE3 para la vía 0 y A6/WD6/WE6 para la vía 1.

Igual que en el pipeline de partida, la escritura ocurre en el flanco de bajada del reloj. Aparece un caso nuevo: ambas vías pueden escribir *el mismo* registro en el mismo ciclo (dependencia WAW dentro del par). Como la vía 1 contiene siempre la instrucción más joven, su valor es el que debe prevalecer: el puerto A6 se escribe después de A3 y, por lo tanto, tiene prioridad.

3.3 Las dos vías de Execute, Memory y Writeback

Cada vía tiene sus propios registros ID/EX, EX/MEM y MEM/WB, su ALU y su multiplexor de *ResultW*. El sumador de *PCTargetE*, en cambio, no se replica: forma parte de la Branch Unit compartida (sección 4.2). La Tabla 2 detalla el contenido de cada registro, que coincide con el del pipeline de partida, replicado por vía (sufijo k). Los registros de control (en azul) también se replican: cada vía lleva sus propias señales *RegWrite*, *ResultSrc*, *MemWrite*, *Jump*, *Branch*, *ALUControl* y *ALUSrc*.

Tabla 2 **Registros de pipeline de cada vía ($k = 0, 1$)**

Registro	Contenido
IF/ID (uno para el par)	$InstrD_0$, $InstrD_1$, PCD_0, PCD_1, $PCPlus4D_0$, $PCPlus4D_1$
ID/EX_k	*RD1*, *RD2*, *PC*, *Rs1*, *Rs2*, *Rd*, *ImmExt*, *PCPlus4*
EX/MEM_k	*ALUResult*, *WriteData*, *Rd*, *PCPlus4*
MEM/WB_k	*ALUResult*, *ReadData*, *Rd*, *PCPlus4*

Ejemplo HDL 1 Memoria de instrucciones con dos puertos y escritura del register file

```
// instruction memory: two consecutive words per cycle
assign rd1 = RAM[a[31:2]];      // InstrF_0 = mem[PCF]
assign rd2 = RAM[a[31:2]+1];    // InstrF_1 = mem[PCF+4]

// register file: two write ports, written in the first half of the cycle
always @(negedge clk) begin
  if (we3) rf[a3] <= wd3;       // lane 0
  if (we6) rf[a6] <= wd6;       // lane 1: same register -> the younger one wins
end
```

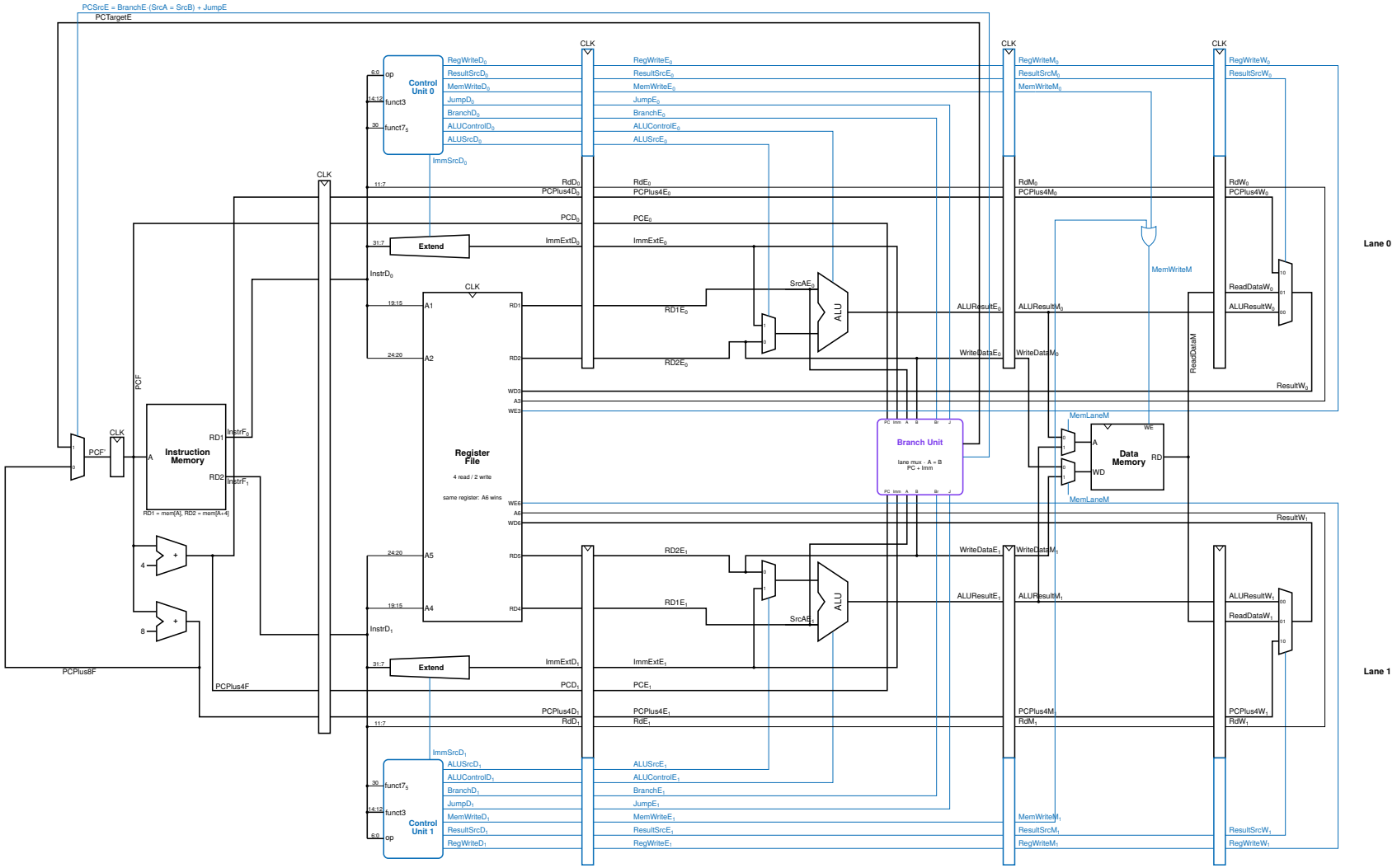


Figura 3 **Paso 1: datapath de dos vías con fetch y decode dobles, register file de cuatro lecturas y dos escrituras, memoria de datos compartida y Branch Unit compartida. Aún sin manejo de hazards**

4 Recursos compartidos

Ambas vías pueden recibir cualquier tipo de instrucción, por lo que cada recurso compartido necesita multiplexores que lo asignen a la vía que lo requiere. Esto funciona porque la Issue Unit (sección 6) garantiza que un mismo par *nunca* contiene dos instrucciones de memoria ni dos saltos.

4.1 Un solo puerto de memoria de datos

La Fig. 4 muestra la etapa Memory. Dos multiplexores frente a la memoria seleccionan la dirección ($ALUResultM_k$) y el dato de escritura ($WriteDataM_k$) de la vía que contiene la instrucción de memoria. Su señal de selección, *MemLaneM*, vale 1 cuando la vía 1 contiene un `lw` o un `sw`; en cualquier otro caso, el puerto se asigna a la vía 0. La señal de escritura es el OR de ambas vías, ya que como máximo una de ellas escribe.

La lectura no requiere selección: *ReadDataM* llega a los registros MEM/WB de *ambas* vías, y solo la vía que contiene el `lw` lo utiliza, por ser la única con $ResultSrcW = 01$. En consecuencia, un par `lw` + ALU se emite igual que un par ALU + `lw`: el orden dentro del par es indiferente.

Ejemplo HDL 2 Puerto único de la memoria de datos

```
assign MemWriteM = MemWriteM_0 | MemWriteM_1;              // at most one lane writes
assign MemLaneM  = MemWriteM_1 | (ResultSrcM_1 == 2'b01);  // lane 1 holds the lw/sw

mux2 #(32) memaddrmux(.d0(ALUResultM_0), .d1(ALUResultM_1), .s(MemLaneM), .y(ALUResultM));
mux2 #(32) memdatamux(.d0(WriteDataM_0), .d1(WriteDataM_1), .s(MemLaneM), .y(WriteDataM));
```

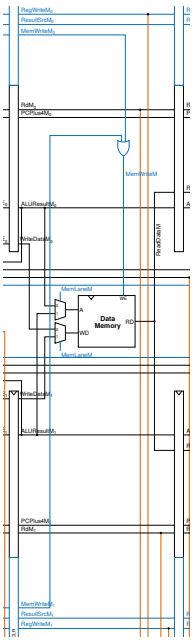


Figura 4 **Puerto único de la memoria de datos y su asignación a una vía**

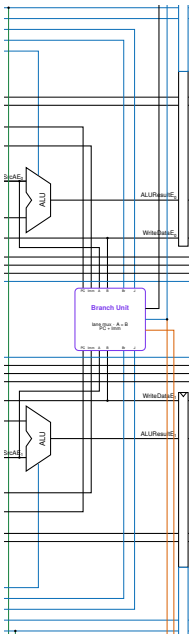


Figura 5 **Branch Unit: recibe los operandos de ambas vías y genera *PCSrcE*, $TakenE_0$ y *PCTargetE***

4.2 La Branch Unit

En el pipeline de partida, el salto se resuelve en Execute mediante $PCSrcE = BranchE \cdot ZeroE + JumpE$: la ALU compara los operandos del `beq` (señal *Zero*) y un sumador dedicado calcula

PCTargetE. Replicar esta lógica en cada vía produciría dos unidades de resolución, de las cuales nunca se usaría más de una por ciclo. En su lugar, agregamos un módulo único, la **Branch Unit**, ubicado en el eje central del datapath (Fig. 5). Tiene tres partes:

- **Selección de la vía.** Como la Issue Unit nunca emite dos saltos en el mismo par, a lo sumo una vía tiene activa *Branch* o *Jump*. La señal $BrLaneE = BranchE_1 + JumpE_1$ controla cuatro multiplexores que toman los operandos de esa vía: *PCE*, *ImmExtE*, *SrcAE* y *WriteDataE* (el valor de *rs2*). Los dos últimos se toman después de los multiplexores de forwarding, de modo que el salto compara valores actualizados.
- **Resolución.** Un único comparador evalúa $SrcA = SrcB$, que reemplaza a la señal *Zero* de la ALU, y un único sumador calcula $PCTargetE = PCE + ImmExtE$. Las ALU dejan de participar en los saltos.
- **Salidas.** *PCSrcE* y *PCTargetE* van a la etapa Fetch, y *PCSrcE* también a la Hazard Unit. Además, $TakenE_0 = PCSrcE \cdot \overline{BrLaneE}$ indica que el salto tomado está en la vía 0; la Hazard Unit usa esta señal para anular la otra instrucción del par (sección 7.2).

Como las señales *Branch* y *Jump* de las dos vías nunca están activas a la vez, basta un OR para combinarlas, igual que la señal de escritura de la memoria de datos.

Ejemplo HDL 3 Branch Unit (`branchunit.v`)

```
wire BrLaneE = BranchE_1 | JumpE_1;             // lane that holds the branch/jump

// operands of that lane (SrcAE / WriteDataE already forwarded)
mux2 #(32) pcmux (PCE_0,        PCE_1,        BrLaneE, PCE);
mux2 #(32) immmux(ImmExtE_0,    ImmExtE_1,    BrLaneE, ImmExtE);
mux2 #(32) amux  (SrcAE_0,      SrcAE_1,      BrLaneE, SrcAE);
mux2 #(32) bmux  (WriteDataE_0, WriteDataE_1, BrLaneE, SrcBE);

assign EqualE    = (SrcAE == SrcBE);            // one comparator
adder pcaddbranch(PCE, ImmExtE, PCTargetE);     // one target adder

assign PCSrcE    = ((BranchE_0 | BranchE_1) & EqualE) | JumpE_0 | JumpE_1;
assign TakenE_0  = PCSrcE & ~BrLaneE;           // taken branch/jump in lane 0
```

5 Hazards de datos entre vías: forwarding

El datapath de la Fig. 3 presenta los mismos hazards RAW que el pipeline de partida, pero con más casos: una instrucción puede necesitar un resultado que aún está en Memory o Writeback de su propia vía *o de la otra*. La solución es la misma: adelantar el valor desde la etapa en que se encuentra. La diferencia es que ahora hay cuatro fuentes posibles en lugar de dos: $ALUResultM_0$, $ALUResultM_1$, $ResultW_0$ y $ResultW_1$.

Estas cuatro señales forman un *bus de forwarding* vertical que cruza el eje central (Fig. 6) y llega a ambas vías. Los multiplexores frente a cada ALU pasan de tres a cinco entradas, y *ForwardAE* y *ForwardBE* pasan de dos a tres bits (Tabla 3). La Fig. 7 muestra el datapath con este cambio. La Hazard Unit aparece en naranja, por ahora solo con sus entradas y sus salidas de forwarding.

Prioridad. En el pipeline de partida, si ambas etapas contienen el registro buscado, tiene prioridad Memory, porque guarda el valor más reciente. Con dos vías se agrega una segunda regla: *dentro de una misma etapa tiene prioridad la vía 1*, porque contiene la instrucción más joven del par. El orden resultante es el de la tabla: M_1, M_0, W_1, W_0. Por ejemplo, consideremos el siguiente código:

Tabla 3 **Códigos de $ForwardAE_k$ y $ForwardBE_k$**

Código	Fuente	Condición (en orden de prioridad)
100	$ALUResultM_1$	$Rs = RdM_1$ y $RegWriteM_1$
011	$ALUResultM_0$	$Rs = RdM_0$ y $RegWriteM_0$
010	$ResultW_1$	$Rs = RdW_1$ y $RegWriteW_1$
001	$ResultW_0$	$Rs = RdW_0$ y $RegWriteW_0$
000	register file ($RD1E/RD2E$)	ninguna de las anteriores, o $Rs =$ x0

```
addi x3, x0, 10   # par: vía 0
addi x3, x0, 30   # par: vía 1
add x4, x3, x0
nop
addi x5, x3, 1
```

El add encuentra x3 en ambas vías de Memory y debe tomar el valor 30, de la vía 1. Dos instrucciones después, addi x5 encuentra x3 en ambas vías de Writeback y vuelve a seleccionar la vía 1, de modo que x5 = 31. Es la misma regla que aplica el register file al escribir (A6 tiene prioridad sobre A3).

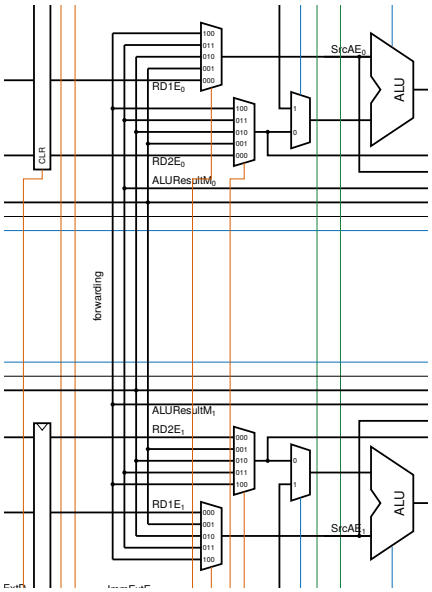


Figura 6 **Bus de forwarding entre vías: las cuatro fuentes (W0, W1, M0, M1) llegan a los multiplexores de cinco entradas de ambas vías**

Ejemplo HDL 4 Selección de la fuente de forwarding

```
// youngest producer first: MEM before WB and, inside a stage, lane 1 before lane 0
function [2:0] fwd(input [4:0] rs, input [4:0] rdM1, rdM0, rdW1, rdW0,
                   input       weM1, weM0, weW1, weW0);
  if      (rs == 5'b0)            fwd = 3'b000;
  else if ((rs == rdM1) & weM1)   fwd = 3'b100;
  else if ((rs == rdM0) & weM0)   fwd = 3'b011;
  else if ((rs == rdW1) & weW1)   fwd = 3'b010;
  else if ((rs == rdW0) & weW0)   fwd = 3'b001;
  else                            fwd = 3'b000;
endfunction
```

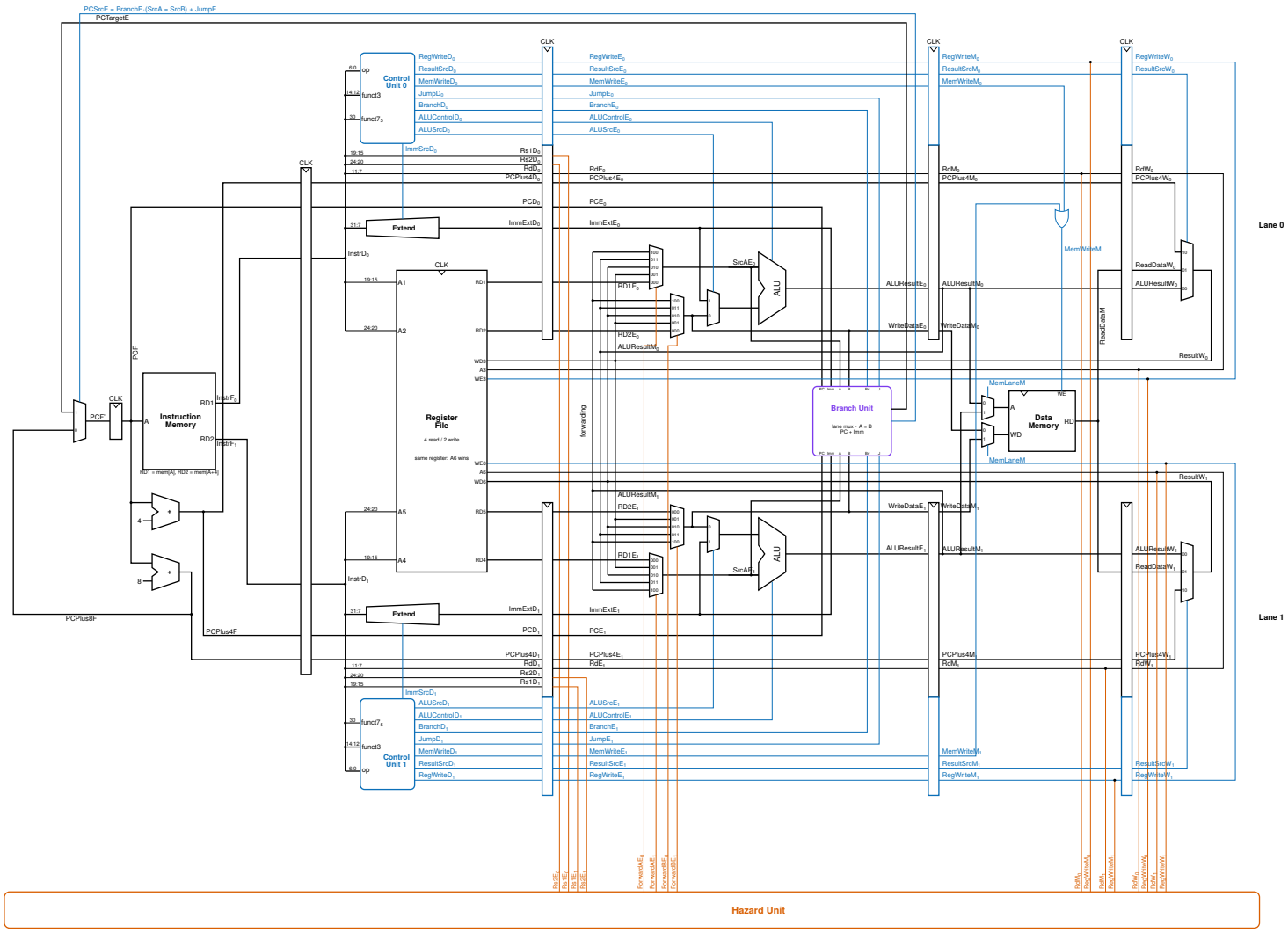


Figura 7 **Paso 2: forwarding entre vías. Bus de cuatro fuentes, multiplexores de cinco entradas y Hazard Unit (solo forwarding)**

6 La Issue Unit

6.1 Dependencias que el forwarding no resuelve

El forwarding adelanta un resultado desde una etapa *posterior*. Sin embargo, las dos instrucciones de un par se encuentran en la *misma* etapa: si el slot 1 necesita el resultado del slot 0, ese valor se calcula en la otra ALU en el mismo ciclo en que el slot 1 lo requiere, y no existe una etapa posterior desde la cual adelantarlo. Algo similar ocurre con los recursos compartidos de la sección 4: dos accesos a memoria o dos saltos en el mismo par no pueden atenderse en un solo ciclo.

En estos casos, la instrucción más joven no puede emitirse en ese ciclo. La decisión la toma un bloque nuevo, la **Issue Unit** (en verde en las figuras).

6.2 Definición

La Issue Unit es un bloque combinacional que, en Decode, examina el par y las instrucciones que se encuentran en Execute, y determina cuántas instrucciones del par se emiten:

- **dos**: el caso normal;
- **una** (solo el slot 0): el slot 1 tiene un conflicto con el slot 0 o depende de un `lw` que está en Execute. Se activa *IssueOneD*;
- **ninguna**: el slot 0 depende de un `lw` que está en Execute. Se activa $LoadUseD_0$ y la Hazard Unit detiene el par completo (sección 7).

Sus entradas son las dos instrucciones del par ($InstrD_0$, $InstrD_1$) y, para detectar el load-use, el registro destino y el bit $ResultSrcE[0]$ (igual a 1 para un `lw`) de cada vía de Execute. Sus dos salidas van a la Hazard Unit, que las convierte en las señales de control del pipeline.

La Issue Unit decodifica por su cuenta qué registros usa *realmente* cada instrucción (Tabla 4). Si comparara los campos *rs1* y *rs2* sin considerar el tipo de instrucción, detectaría dependencias falsas, y cada una dividiría un par innecesariamente. Por ejemplo, los bits 24:20 de un `addi` forman parte del inmediato y no designan un registro: sin esta tabla, un `addi` cuyo inmediato coincidiera con el *rd* del slot 0 se trataría como dependiente.

Tabla 4 **Registros que usa cada tipo de instrucción, según la Issue Unit**

Instrucción	lee *rs1*	lee *rs2*	escribe *rd*
tipo R (`add`, `sub`, `and`, `or`, `xor`, `slt`)	sí	sí	sí
tipo I (`addi`, ...)	sí	no	sí
`lw`	sí	no	sí
`sw`	sí	sí	no
`beq`	sí	sí	no
`jal`	no	no	sí

6.3 Condiciones de emisión

Con esa decodificación, la Issue Unit evalúa cuatro condiciones para el slot 1 y una para el slot 0 (Tabla 5).

Conviene notar tres cosas:

- Solo importa el RAW del slot 1 respecto del slot 0; el caso inverso no puede ocurrir, porque el slot 0 es más antiguo. Las dependencias WAR y WAW dentro del par no impiden la emisión. WAR no genera conflicto porque ambos slots leen sus operandos antes de que

Tabla 5 **Condiciones de la Issue Unit**

Señal	Se activa cuando	Ejemplo
rawD	el slot 0 escribe un registro ($\neq$ `x0`) que el slot 1 lee (RAW dentro del par)	`addi x1,x0,5` / `addi x2,x1,1`
memD	ambos slots acceden a memoria (un solo puerto)	`sw` / `sw`, `lw` / `sw`
ctrlD	ambos slots son saltos (una sola Branch Unit)	`beq` / `beq`, `beq` / `jal`
$luseD_1$	el slot 1 lee el destino de un `lw` que está en Execute (en cualquier vía)	`lw x4` en EX; slot 1 = `add x5,x3,x4`
IssueOneD	$rawD + memD + ctrlD + luseD_1$	se emite solo el slot 0
$LoadUseD_0$	el slot 0 lee el destino de un `lw` que está en Execute (en cualquier vía)	no se emite ninguna

cualquiera escriba, y WAW queda resuelta por la prioridad de la vía 1 en el register file y en el forwarding.

- *memD* y *ctrlD* solo se activan si *ambos* slots usan el recurso. Una instrucción de memoria o un salto puede emitirse en cualquiera de los dos slots junto con una instrucción ALU, en cualquier orden; los multiplexores de la sección 4 asignan el recurso a la vía correspondiente.
- Si el slot 1 depende de un `lw` en Execute, el slot 0 puede avanzar ($luseD_1$ solo divide el par). Si el que depende es el slot 0, no se emite ninguna instrucción, porque el slot 1 no puede adelantarse a él.

Ejemplo HDL 5 Condiciones de la Issue Unit

```
// RAW inside the pair: slot 1 needs the result of slot 0 in the same cycle (cannot forward)
wire rawD   = writes_0 & ((useRs1_1 & (rs1_1 == rd_0)) | (useRs2_1 & (rs2_1 == rd_0)));
// structural: one data memory port, one branch unit
wire memD   = mem_0  & mem_1;
wire ctrlD  = ctrl_0 & ctrl_1;

// load-use with a lw in Execute (either lane)
wire luseD_1 = loaduse(rs1_1, rs2_1, useRs1_1, useRs2_1, RdE_0, RdE_1, ResultSrcEb0_0,
ResultSrcEb0_1);
assign LoadUseD_0 = loaduse(rs1_0, rs2_0, useRs1_0, useRs2_0, RdE_0, RdE_1, ResultSrcEb0_0,
ResultSrcEb0_1);
assign IssueOneD  = rawD | memD | ctrlD | luseD_1;
```

6.4 Emisión parcial y reagrupamiento

Cuando solo se emite el slot 0, el slot 1 no debe perderse ni ejecutarse dos veces: debe permanecer en Decode y emitirse en un ciclo posterior, sin alterar el orden del programa. Hay dos alternativas:

1. detener el par completo y repetirlo con el slot 0 convertido en burbuja;
2. *reagrupar*: la instrucción en espera pasa a ser el slot 0 del siguiente par, que se completa con la instrucción siguiente del programa.

La primera es más simple, pero desperdicia un ciclo de cada vía en cada conflicto. Adoptamos la segunda (Fig. 8). La Hazard Unit genera

$$RegroupD = IssueOneD \cdot \overline{lwStallD} \cdot \overline{PCSrcE},$$

y *RegroupD* controla siete multiplexores en Fetch: seis frente al registro IF/ID (la instrucción, el *PC* y el $PC + 4$ de cada slot) y uno para el siguiente PC:

- **nuevo slot 0** ← el slot 1 actual ($InstrD_1$, PCD_1, $PCPlus4D_1$), realimentado desde la salida de IF/ID;
- **nuevo slot 1** ← $InstrF_0$ = mem[*PCF*], con dirección *PCF* y $PC + 4$ igual a *PCPlus4F*;
- **siguiente PC** ← $PCF + 4$ en lugar de $PCF + 8$.

$InstrF_0$ es justamente la instrucción que sigue al slot 1. Mientras un par (PCD_0, PCD_0+4) está en Decode, Fetch se encuentra en $PCF = PCD_0 + 8 = PCD_1 + 4$. El nuevo par queda formado por (PCD_1, PCD_1+4), es decir, por dos instrucciones consecutivas. Al avanzar el PC solo en 4, Fetch queda en $PCD_1 + 8$, alineado con el nuevo par. La instrucción en espera siempre vuelve a entrar por el slot 0, de modo que la vía 1 sigue conteniendo la más joven. En el mismo ciclo, $FlushE_1$ inserta una burbuja en la vía 1, porque el slot 1 no se emitió.

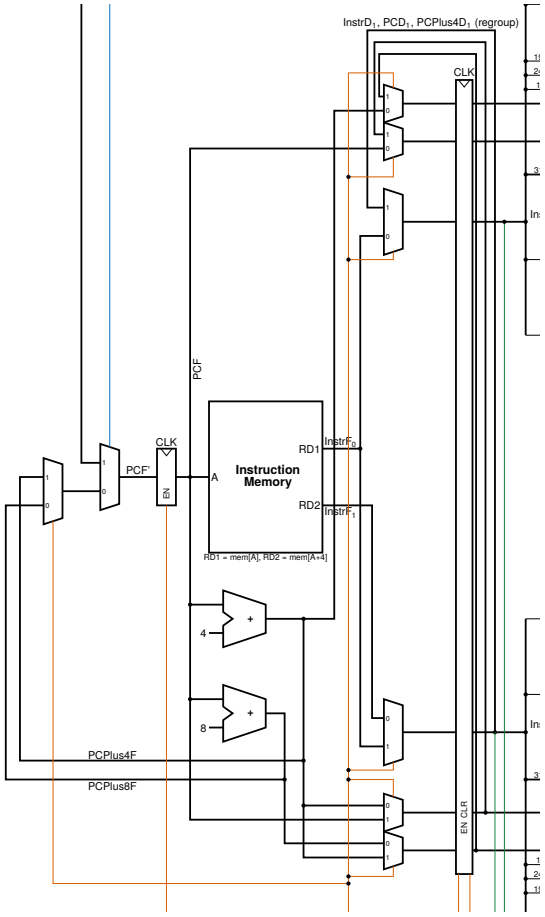


Figura 8 **Fetch con reagrupamiento: *RegroupD* elige entre el par recién buscado y el slot 1 en espera (lazo $InstrD_1$, PCD_1, $PCPlus4D_1$), y entre $PC + 8$ y $PC + 4$**

La Fig. 9 muestra el caso más simple:

- **Ciclo 2.** `addi x2, x1, 1` necesita el valor de `x1` que calcula la otra instrucción del par, por lo que *rawD* la retiene en Decode (celda rayada).
- **Ciclo 3.** Se reagrupa con `addi x3` y el nuevo par se emite completo. `x1` ya está en Memory y llega por forwarding.
- **Ciclo 5.** El par siguiente (`add x4`, `sub x5`) toma sus operandos de ambas vías a la vez.

La Fig. 10 repite el mecanismo varias veces seguidas, con una cadena de incrementos de `x1` intercalada con instrucciones independientes. Cada `sw` guarda el valor actual de `x1`, de modo que una instrucción perdida o duplicada alteraría la secuencia de valores en memoria.

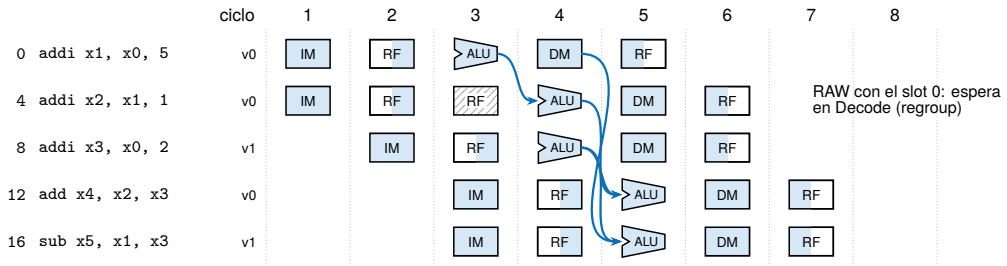


Figura 9 **RAW dentro del par: solo se emite el slot 0, y `addi x2` se reagrupa con la instrucción siguiente**

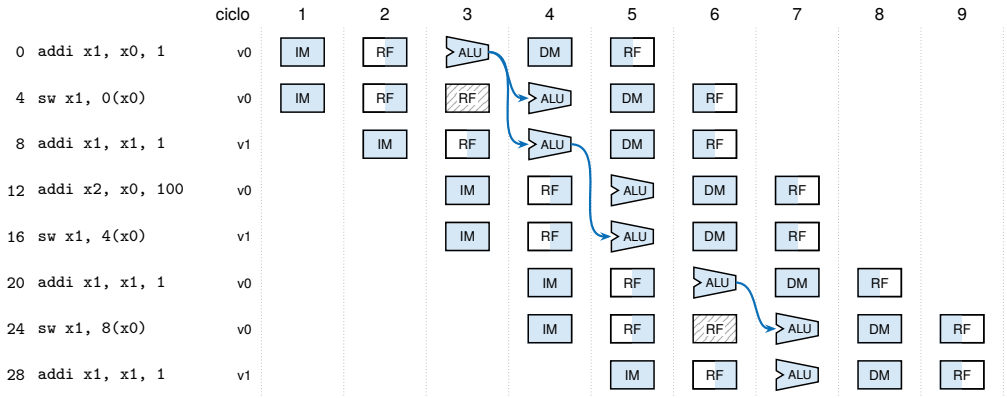


Figura 10 **Emisiones parciales repetidas. Las instrucciones rayadas esperan en Decode y se emiten en la vía 0 del ciclo siguiente**

Ejemplo HDL 6 Selección del siguiente PC y reagrupamiento frente a IF/ID

```
// next PC: PC+8 (pair issued), PC+4 (regroup) or the branch target
mux3 #(32) pcmux(.d0(PCPlus4F), .d1(PCTargetE), .d2(PCPlus8F),
                 .s({~PCSrcE & ~RegroupD, PCSrcE}), .y(PCNextF));

// Regroup: slot 1 (still not executed) becomes the new slot 0 and the
// instruction at PCF (= PCD_1 + 4) becomes the new slot 1.
wire [31:0] InstrN_0   = RegroupD ? InstrD_1   : InstrF_0;
wire [31:0] InstrN_1   = RegroupD ? InstrF_0   : InstrF_1;
wire [31:0] PCN_0      = RegroupD ? PCD_1      : PCF;
wire [31:0] PCN_1      = RegroupD ? PCF        : PCPlus4F;
wire [31:0] PCPlus4N_0 = RegroupD ? PCPlus4D_1 : PCPlus4F;
wire [31:0] PCPlus4N_1 = RegroupD ? PCPlus4F   : PCPlus8F;
```

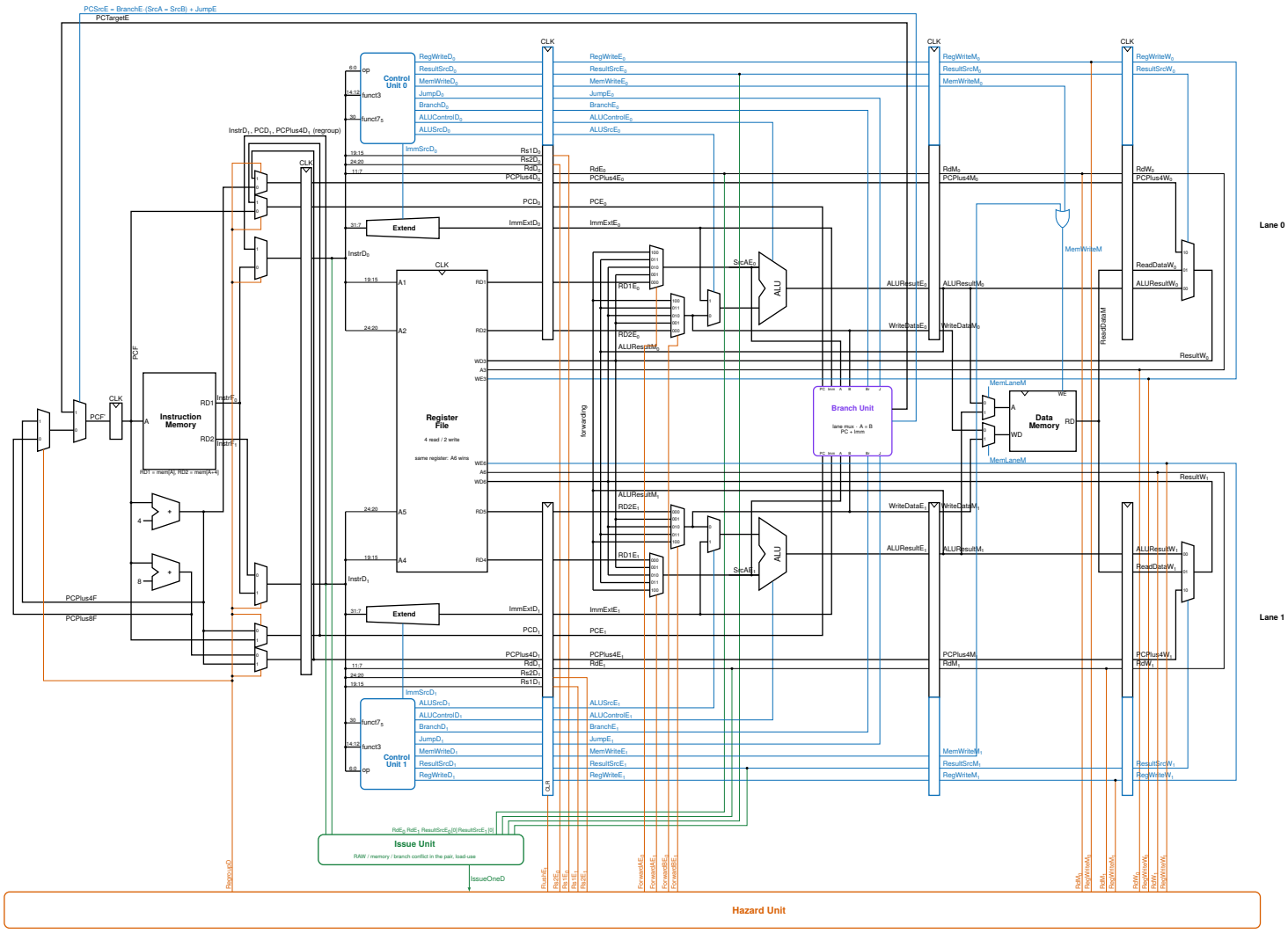


Figura 11 **Paso 3: Issue Unit (verde) y reagrupamiento del par. *IssueOneD* genera *RegroupD* y la burbuja $FlushE_1$ en la vía 1**

7 Stalls y control hazards

Faltan los dos mecanismos del pipeline de partida que el forwarding no cubre: el stall por `lw` y el flush por salto tomado. Ambos se conservan, con ajustes para el par.

7.1 Load-use

Un `lw` entrega su dato al final de Memory, por lo que la instrucción que lo usa a continuación debe esperar un ciclo. Con dos vías, el `lw` puede estar en cualquiera de ellas y el consumidor en cualquiera de los dos slots:

- si el consumidor es el **slot 0** ($LoadUseD_0$), se detiene el par completo: *StallF* y *StallD* congelan el PC y el registro IF/ID, y $FlushE_0$ y $FlushE_1$ insertan una burbuja en ambas vías;
- si el consumidor es solo el **slot 1** ($luseD_1$), el slot 0 se emite y el slot 1 se reagrupa, como se describió en la sección anterior.

La Fig. 12 muestra el primer caso. `lw x4` se ejecuta en la vía 1, junto con un `addi` independiente en la vía 0. El par siguiente comienza con `add x5, x3, x4`, que necesita `x4`. Ambas instrucciones del par esperan un ciclo en Decode, y en el ciclo 5 `x4` llega desde Writeback de la vía 1. `sw x5`, que está en Fetch, también espera un ciclo.

Si el `lw` y su consumidor quedan *en el mismo par*, actúa primero *rawD*: el consumidor se reagrupa como slot 0. En el ciclo siguiente, el `lw` está en Execute y se activa $LoadUseD_0$.

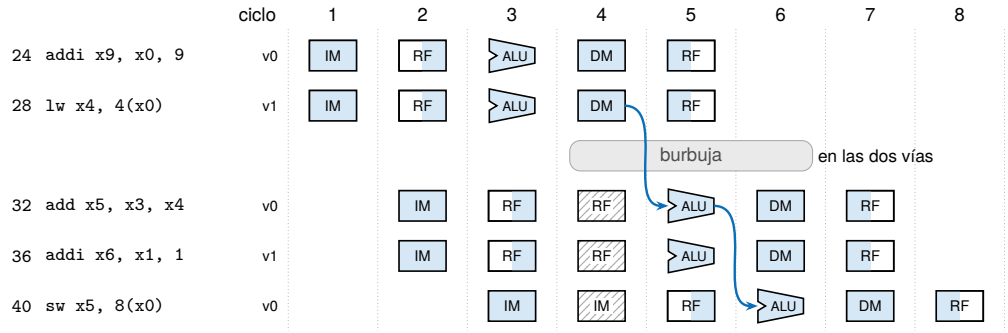


Figura 12 **Load-use con el `lw` en la vía 1: el par completo espera un ciclo y se inserta una burbuja en ambas vías**

7.2 Saltos en cualquier posición del par

Como en el pipeline de partida, el salto se resuelve en Execute y se predice como no tomado. La diferencia es que lo resuelve la Branch Unit, que atiende a la vía en la que se encuentre el salto. Cuando el salto se toma (*PCSrcE*), la Branch Unit entrega la dirección de destino (*PCTargetE*) a Fetch. Las instrucciones buscadas después del salto pertenecen al camino incorrecto y deben descartarse:

- el par que está en Decode, con $FlushE_0$ y $FlushE_1$;
- el par que está en Fetch, con *FlushD*.

Además, el salto puede compartir el par con otra instrucción, y en ese caso el tratamiento depende del slot que ocupa (Fig. 13):

- **Salto en el slot 0** (vía 0): la otra instrucción del par, en la vía 1, es *más joven* y, por lo tanto, también pertenece al camino incorrecto. Como ya está en Execute, se anula al pasar a

Memory mediante $FlushM_1 = TakenE_0$, señal que genera la Branch Unit. Es el único registro EX/MEM que requiere entrada *CLR*.

- **Salto en el slot 1** (vía 1): la otra instrucción del par, en la vía 0, es *más antigua* y debe completarse normalmente. $FlushM_1$ no se activa, porque $TakenE_0 = 0$.

Por último, un salto tomado tiene prioridad sobre el load-use: si el par en Decode pertenece al camino incorrecto, detenerlo no tiene sentido. Por ello la Hazard Unit usa $lwStallD = LoadUseD_0 \cdot \overline{PCSrcE}$. Por la misma razón, *RegroupD* se anula cuando *PCSrcE* está activa.

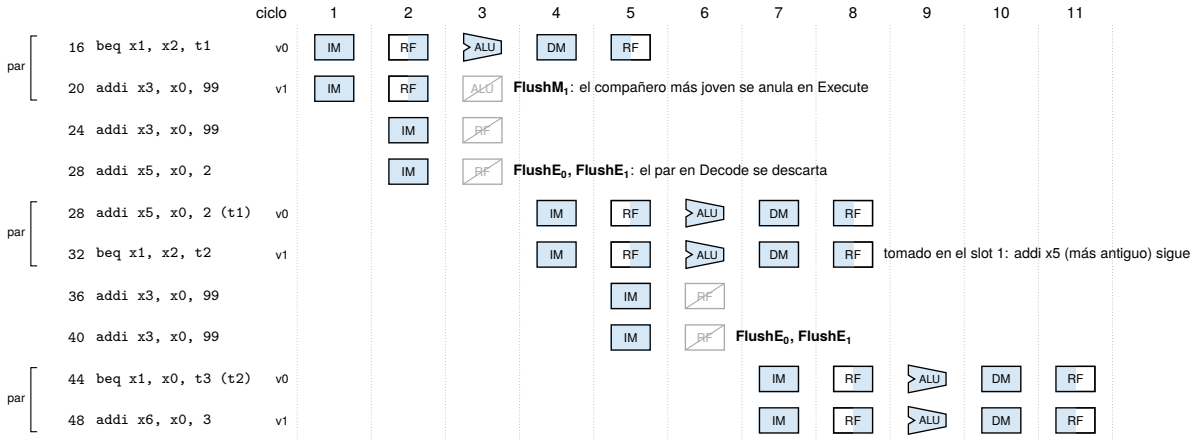


Figura 13 **Saltos tomados en el slot 0 y en el slot 1. Las celdas grises tachadas son instrucciones del camino incorrecto que se descartan; las que escriben 99 en `x3` nunca llegan a Writeback**

7.3 Resumen de la Hazard Unit

La Tabla 6 compara las ecuaciones de la Hazard Unit del pipeline de partida con las del procesador dual-issue. Las señales *PCSrcE* y $TakenE_0$ provienen de la Branch Unit, y el forwarding se calcula igual para ambas vías, con la función de la sección 5. La Fig. 14 muestra el procesador completo.

Tabla 6 **Salidas de la Hazard Unit: pipeline de partida y dual-issue**

Salida	Pipeline de partida	Dual-issue
lwStallD	$ResultSrcE_0 \cdot (Rs1D{=}RdE + Rs2D{=}RdE)$	$LoadUseD_0 \cdot \overline{PCSrcE}$
StallF, *StallD*	*lwStallD*	*lwStallD*
RegroupD	—	$IssueOneD \cdot \overline{lwStallD} \cdot \overline{PCSrcE}$
FlushD	*PCSrcE*	*PCSrcE*
$FlushE_0$	$lwStallD + PCSrcE$	$lwStallD + PCSrcE$
$FlushE_1$	—	$lwStallD + PCSrcE + IssueOneD$
$FlushM_1$	—	$TakenE_0$
ForwardAE/BE	2 bits, 3 fuentes	3 bits, 5 fuentes (Tabla 3)

Ejemplo HDL 7 Stalls, reagrupamiento y flushes de la Hazard Unit

```
assign lwStallD = LoadUseD_0 & ~PCSrcE;          // a taken branch has priority
assign StallD   = lwStallD;
assign StallF   = lwStallD;
assign RegroupD = IssueOneD & ~lwStallD & ~PCSrcE;
```

```
assign FlushD   = PCSrcE;
assign FlushE_0 = lwStallD | PCSrcE;
assign FlushE_1 = lwStallD | PCSrcE | IssueOneD; // bubble also when slot 1 waits
assign FlushM_1 = TakenE_0;                      // taken in lane 0: annul its younger partner
```

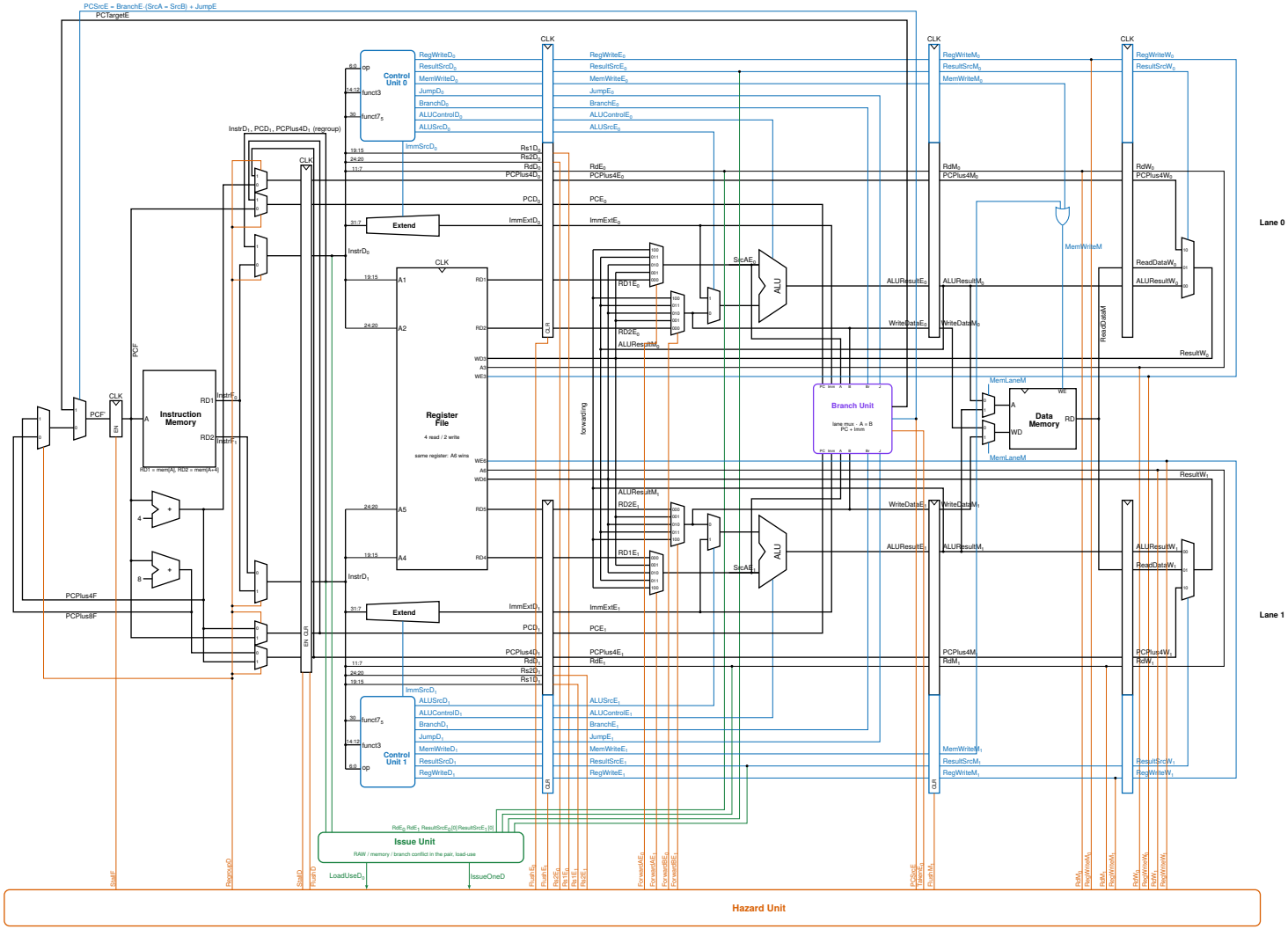


Figura 14 **Procesador dual-issue completo: forwarding entre vías, Issue Unit con reagrupamiento, Branch Unit compartida, stalls por `lw` y flushes por salto en cualquier slot**

8 Resumen

La Tabla 7 resume qué partes del pipeline de partida se replicaron, cuáles se ampliaron, cuáles comparten ambas vías y cuáles son nuevas.

Tabla 7 **Del pipeline de partida al dual-issue**

	Elementos
Replicado	unidad de control, Extend, registros ID/EX, EX/MEM y MEM/WB, ALU, multiplexor de *ResultW*
Ampliado	memoria de instrucciones (2 lecturas), register file (4 lecturas, 2 escrituras), registro IF/ID (un par), multiplexores de forwarding (5 fuentes), Hazard Unit
Compartido	memoria de datos (un puerto, *MemLaneM*) y Branch Unit, nueva (un comparador y un sumador de *PCTargetE*; *PCSrcE*, $TakenE_0$)
Nuevo	Issue Unit, multiplexores de reagrupamiento y selección $PC + 8$ / $PC + 4$, *RegroupD*, $FlushE_1$, $FlushM_1$

El paso del pipeline escalar al procesador dual-issue se basa en tres ideas:

1. **El orden queda fijado por la posición.** El slot 0 va siempre a la vía 0 y contiene la instrucción más antigua, y el reagrupamiento conserva esta propiedad. De ella se derivan todas las prioridades: la vía 1 prevalece en el register file y en el forwarding, y es la única que se anula con $FlushM_1$.
2. **Los recursos compartidos se asignan con multiplexores, no se replican.** La memoria de datos y la Branch Unit atienden a cualquiera de las dos vías, siempre que la Issue Unit garantice que no las solicitan ambas en el mismo ciclo.
3. **Lo que no puede adelantarse se separa.** El forwarding resuelve las dependencias entre ciclos distintos. Las dependencias dentro del par y los conflictos de recursos los resuelve la Issue Unit, que divide el par sin perder ni repetir instrucciones.

Referencias

[1] S. L. Harris y D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.